

Continuous Speech Emotion Recognition

Algorithms and Dataset

1. Algorithms / Pseudocode

Below is simple pseudocode for every block in the diagram.

Pre-processing

```
PREPROCESS(audio):  
    x = resample(audio, 16kHz)  
    x = normalize(x)  
    return x
```

Feature extraction (Mel / MFCC)

```
EXTRACT_FEATURES(window):  
    mel = mel_spectrogram(window)  
    mel = log(mel)  
    mel = normalize(mel)  
    return mel
```

CNN + Bi-LSTM

```
CLASSIFY(mel):  
    h = CNN(mel)  
    h = BiLSTM(h)  
    scores = softmax(Dense(h))  
    label = argmax(scores)  
    conf = max(scores)  
    return label, conf
```

Window-wise emotion detection [Contribution]

```
WINDOW_WISE_DETECT(audio):  
    for t = 0, 1, 2, ... until audio ends:  
        w = audio from t to t+2 sec  
        feats = EXTRACT_FEATURES(w)  
        label[t], conf[t] = CLASSIFY(feats)  
    return all (t, label, conf)
```

Smoothing weak / jumping predictions [Contribution]

```
SMOOTH(labels, conf):  
    for each window i:  
        if labels[i] differs from both neighbours  
        and conf[i] is low:  
            labels[i] = majority of nearby windows  
        else:  
            labels[i] = majority of nearby windows  
    return labels
```

Finding emotion change points [Contribution]

```
FIND_CHANGES(labels):  
    changes = empty  
    for i = 2 to N:  
        if labels[i] != labels[i-1]:  
            changes.append(i)  
    return changes
```

Grouping into stable emotion regions [Contribution]

```
GROUP_STABLE(times, labels):  
    start = times[1]  
    cur = labels[1]  
    regions = empty  
    for i = 2 to N:  
        if labels[i] != cur:  
            regions.append(Stable(start, times[i], cur))  
            start = times[i]  
            cur = labels[i]  
    regions.append(Stable(start, end_time, cur))
```

```
return regions
```

Marking transition intervals [Contribution]

```
MARK_TRANSITIONS(regions):  
    timeline = empty  
    for each next pair of stable regions A then B:  
        timeline.add(A)  
        timeline.add(Transition from A.emotion to B.emotion)  
    timeline.add(last region)  
    return timeline
```

Continuous emotion timeline output [Contribution]

```
MAKE_TIMELINE(timeline):  
    for each item:  
        print start-end : emotion (stable)  
        or start-end : Transition (old -> new)  
    return final timeline
```

Checking with dataset labels

```
EVALUATE(window_preds, meld_labels):  
    for each utterance with gold label g:  
        take window predictions inside that utterance  
        p = majority vote  
        compare p with g  
    report Accuracy and F1
```

2. Dataset

We use MELD (Multimodal EmotionLines Dataset). It has Friends TV show dialogues with emotion labels and can be downloaded without special college permission.

Download links:

- 1) Official page: <https://affective-meld.github.io/>
- 2) Raw data: <https://huggingface.co/datasets/declare-lab/MELD/resolve/main/MELD.Raw.tar.gz>
- 3) Alternate raw link: <https://web.eecs.umich.edu/~mihalcea/downloads/MELD.Raw.tar.gz>
- 4) Labels and readme: <https://github.com/declare-lab/MELD>

MELD has about 1433 dialogues and more than 13000 utterances. Each utterance has audio, text, speaker name and one emotion. The emotion classes are anger, disgust, fear, joy, neutral, sadness and surprise. Sentiment labels (positive, negative, neutral) are also given. Train has about 9989 utterances, development about 1109 and test about 2610.

In our system we use the audio part for prediction. The MELD emotion labels are used to train the model and to check accuracy. The continuous timeline itself is made by our contribution steps. Because labels are for whole utterances, we compare the majority of window predictions inside an utterance with its gold label.

Reference: S. Poria, D. Hazarika, N. Majumder, G. Naik, E. Cambria and R. Mihalcea, MELD: A Multimodal Multi-Party Dataset for Emotion Recognition in Conversations, Proceedings of ACL, pp. 527-536, 2019.